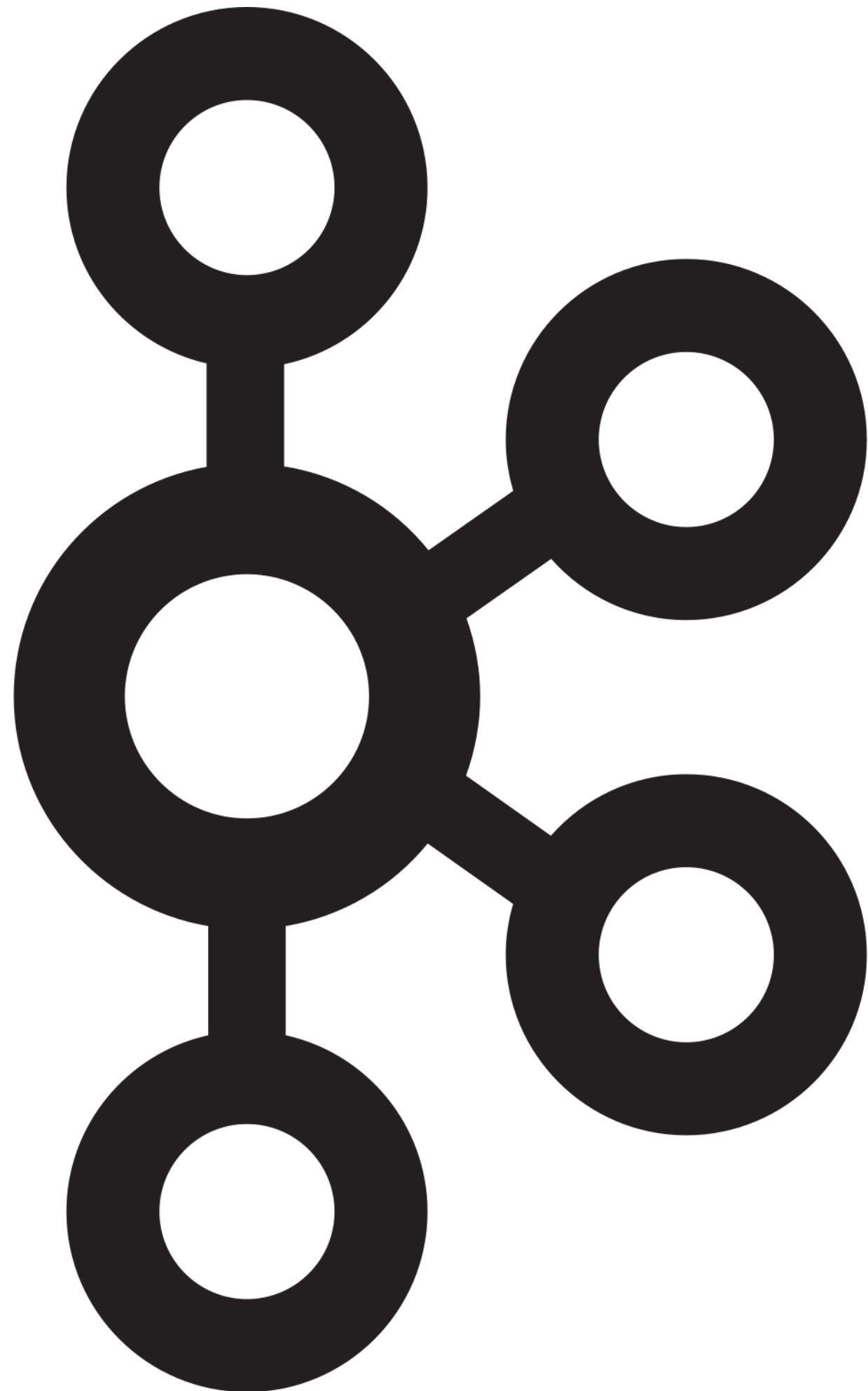


Kafka Cluster

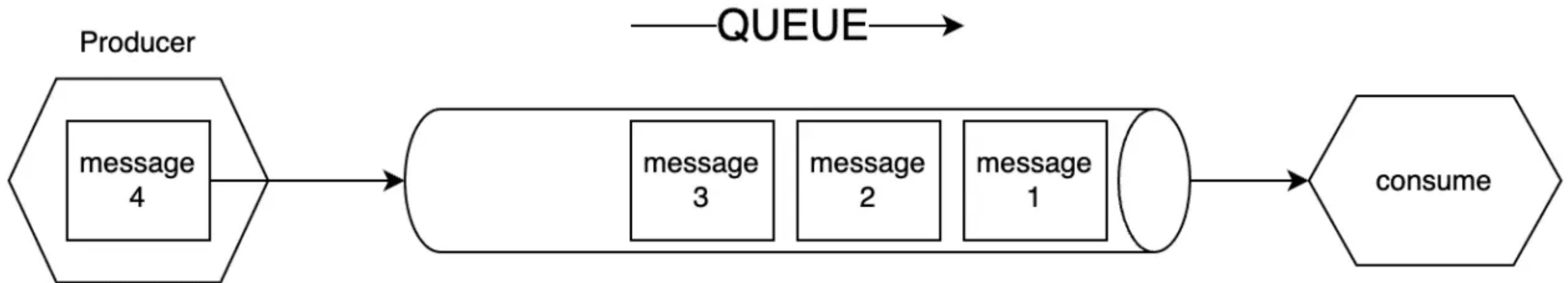
Очередь сообщений



Apache Kafka — распределённый программный брокер сообщений с открытым исходным кодом, разрабатываемый в рамках фонда Apache на языках Java и Scala.

Цель проекта — создание горизонтально масштабируемой платформы для обработки потоковых данных в реальном времени с высокой пропускной способностью и низкой задержкой.

Очередь сообщений

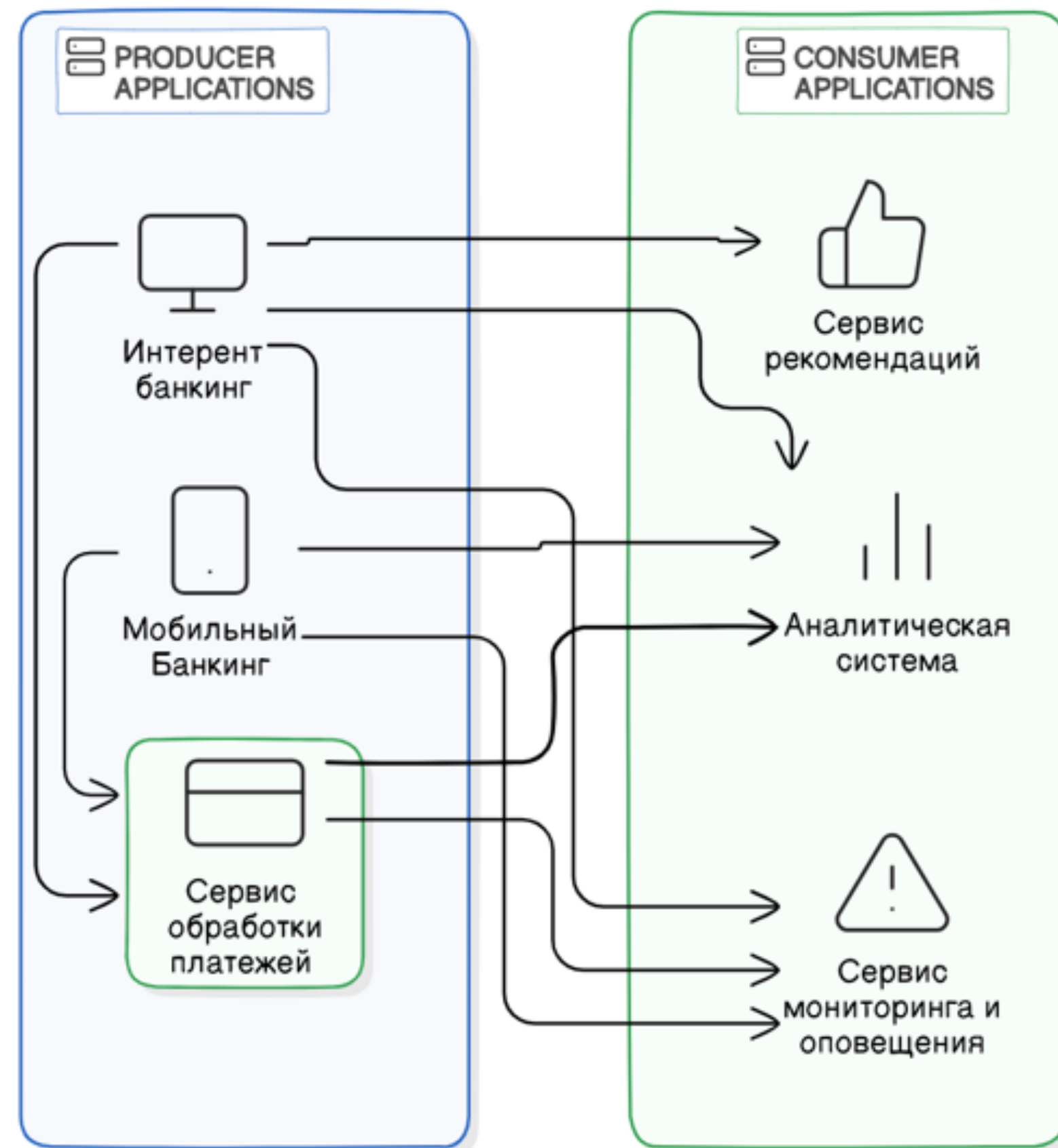


В процессе стриминга данных используются основные три участника обработки данных:

- Producer - тот, кто создает данные и направляет данные в очередь
- Queue - очередь сообщений
- Consumer - тот, кто принимает сообщения из очереди

Средство взаимодействия микросервисов

Без централизованной
очереди сообщений



С централизованной
очередью сообщений

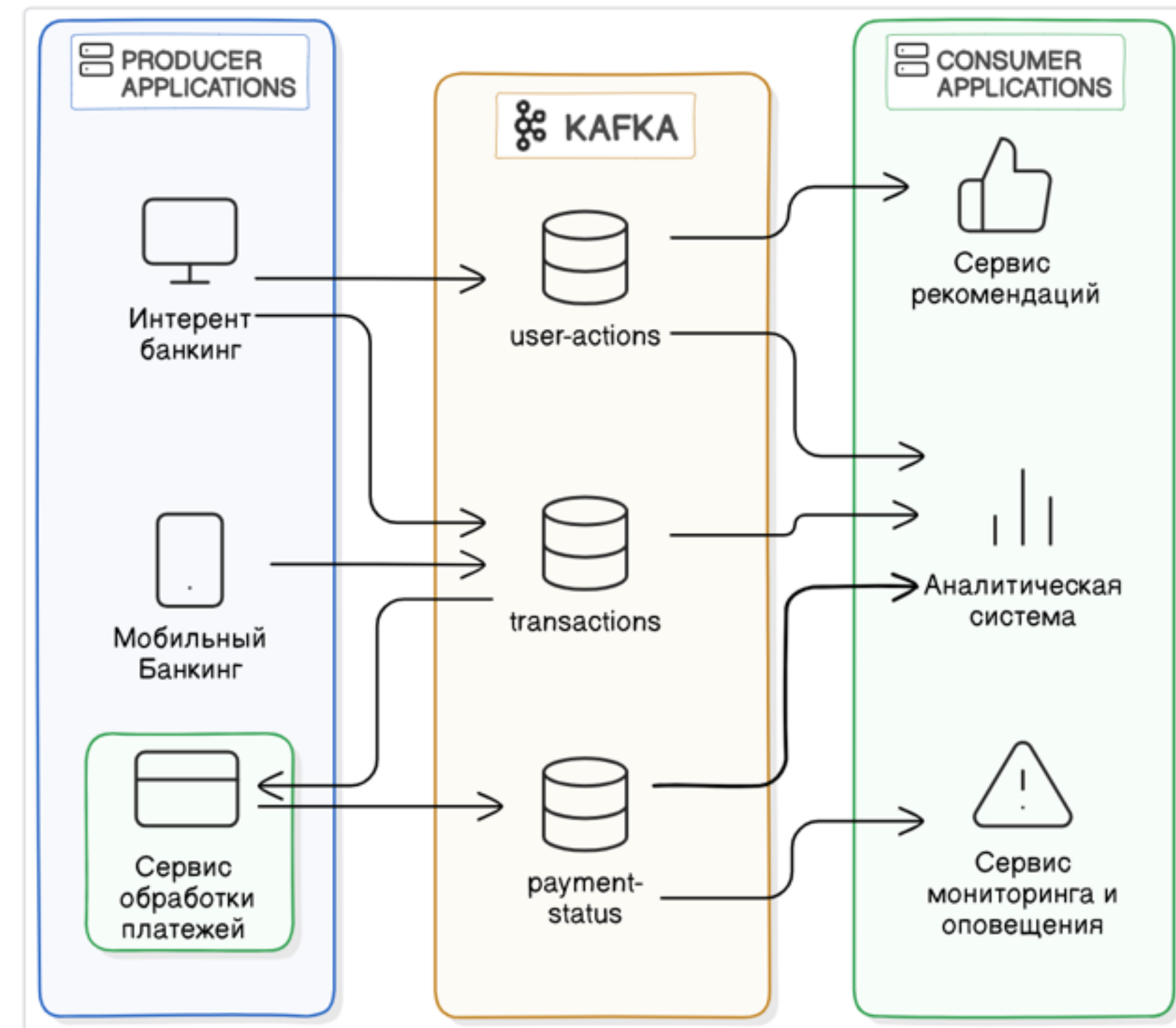
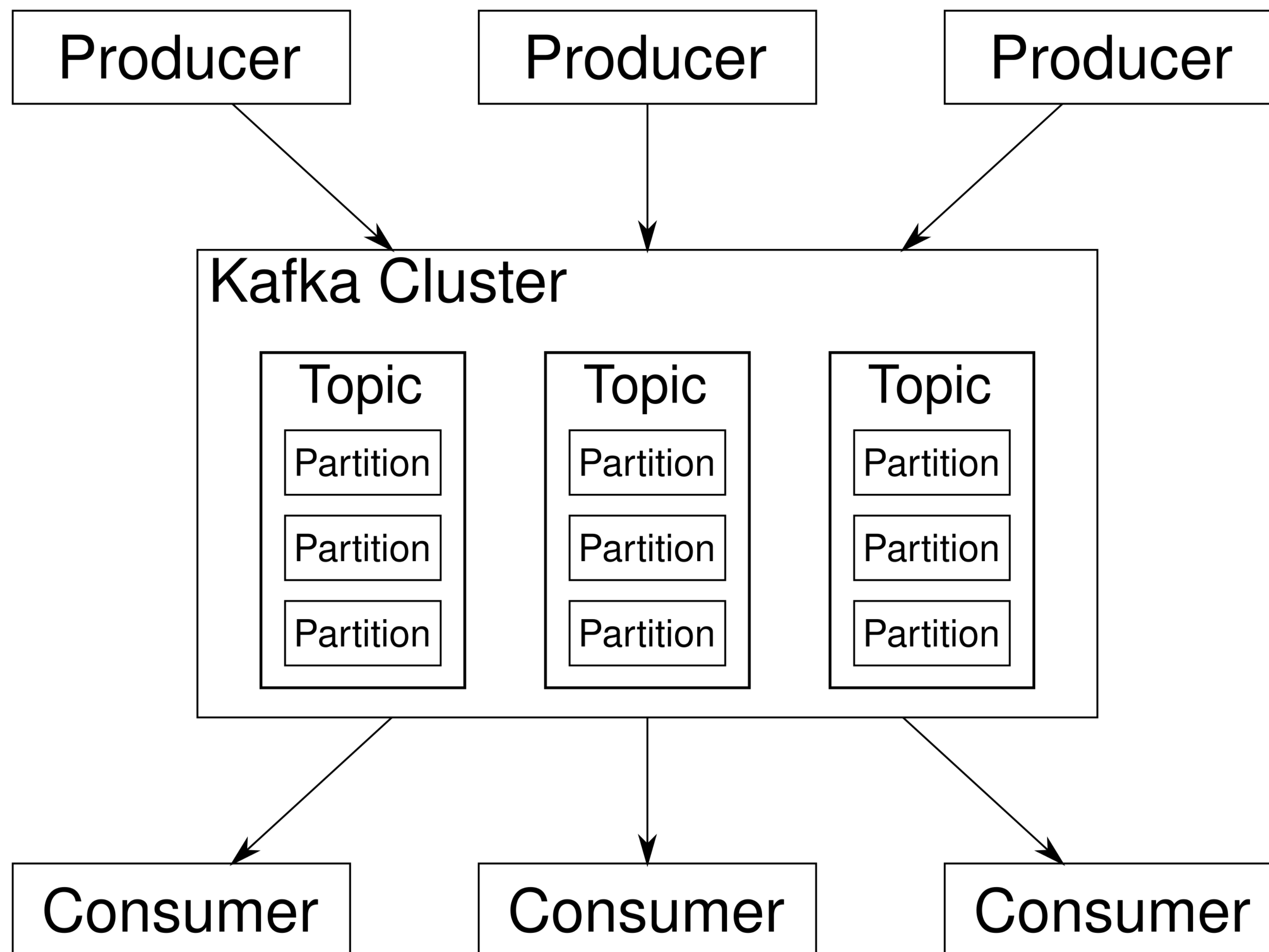


Схема Kafka Cluster

ОСНОВНЫЕ ПОНЯТИЯ
Kafka Cluster:

- Topic
- Partition
- Offset

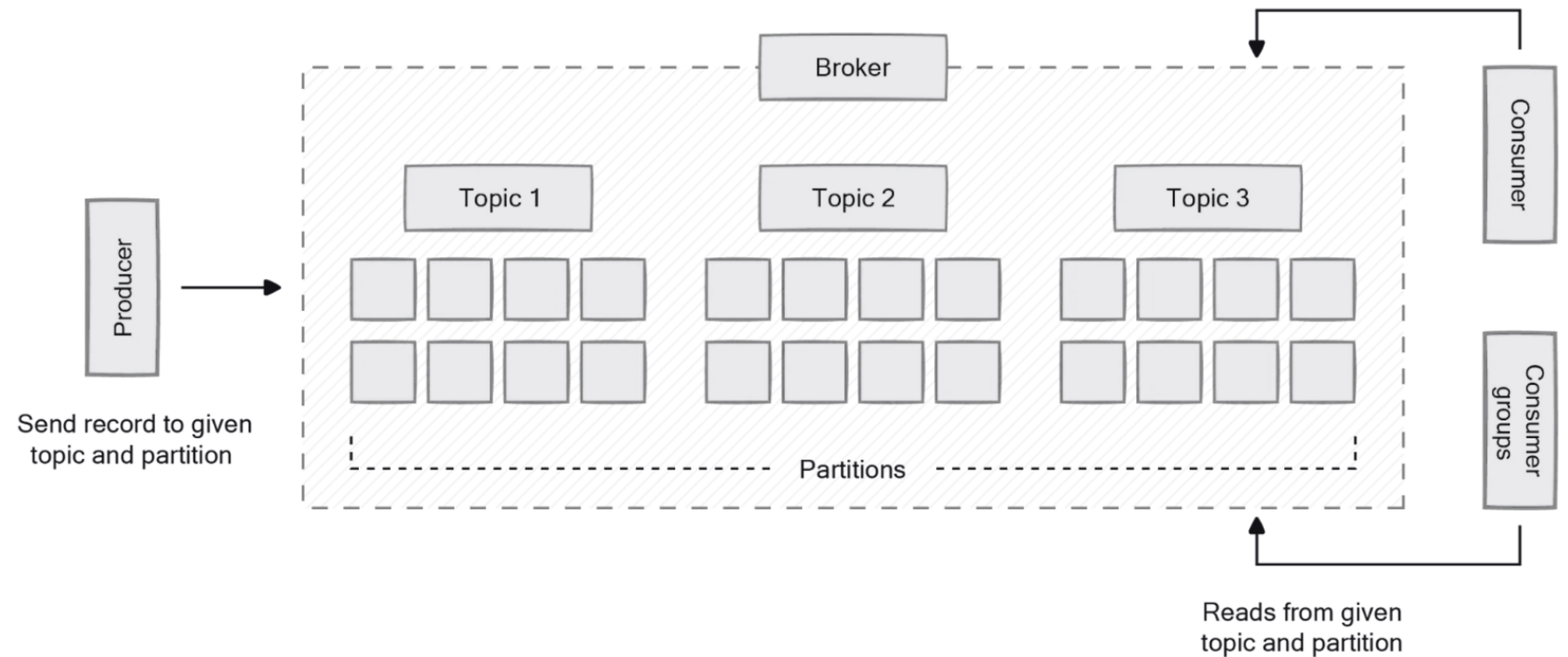


Kafka. Топик

Топик в Kafka — это логическая единица хранения сообщений.

Каждый топик представляет собой категорию или поток данных:

- производители (producers) отправляют в топик свои сообщения
- потребители (consumers, консьюмеры) читают сообщения из топика.



Kafka. Партиция

Партиция — это физическая единица хранения сообщений внутри топика. Партиция разбивает топик на несколько логически независимых частей, где каждая представляет собой упорядоченный, неизменяемый набор записей. Сообщения добавляются только в конец партиции.

Для чего нужны партиции:

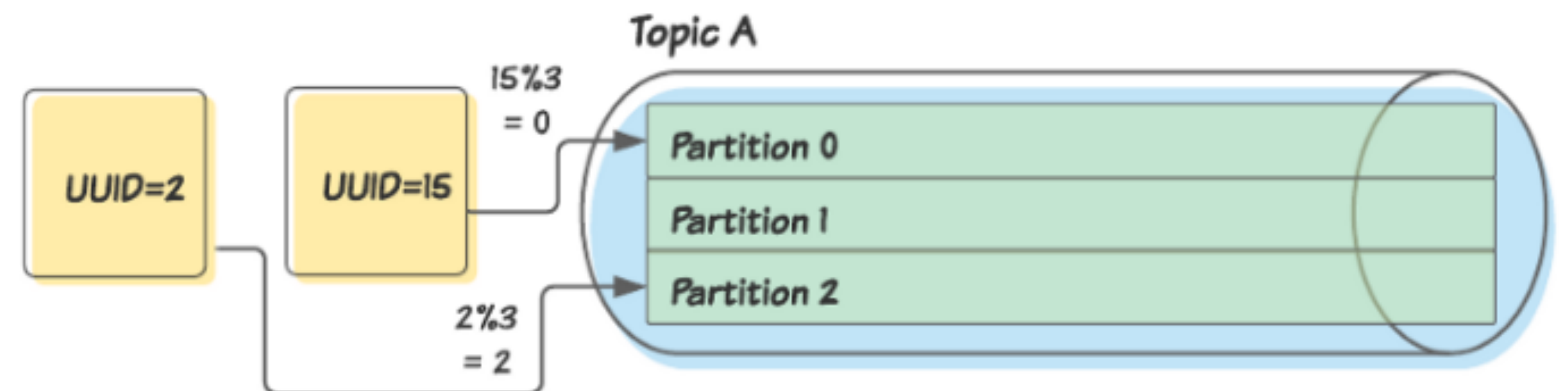
- Параллелизм
- Масштабируемость
- Отказоустойчивость



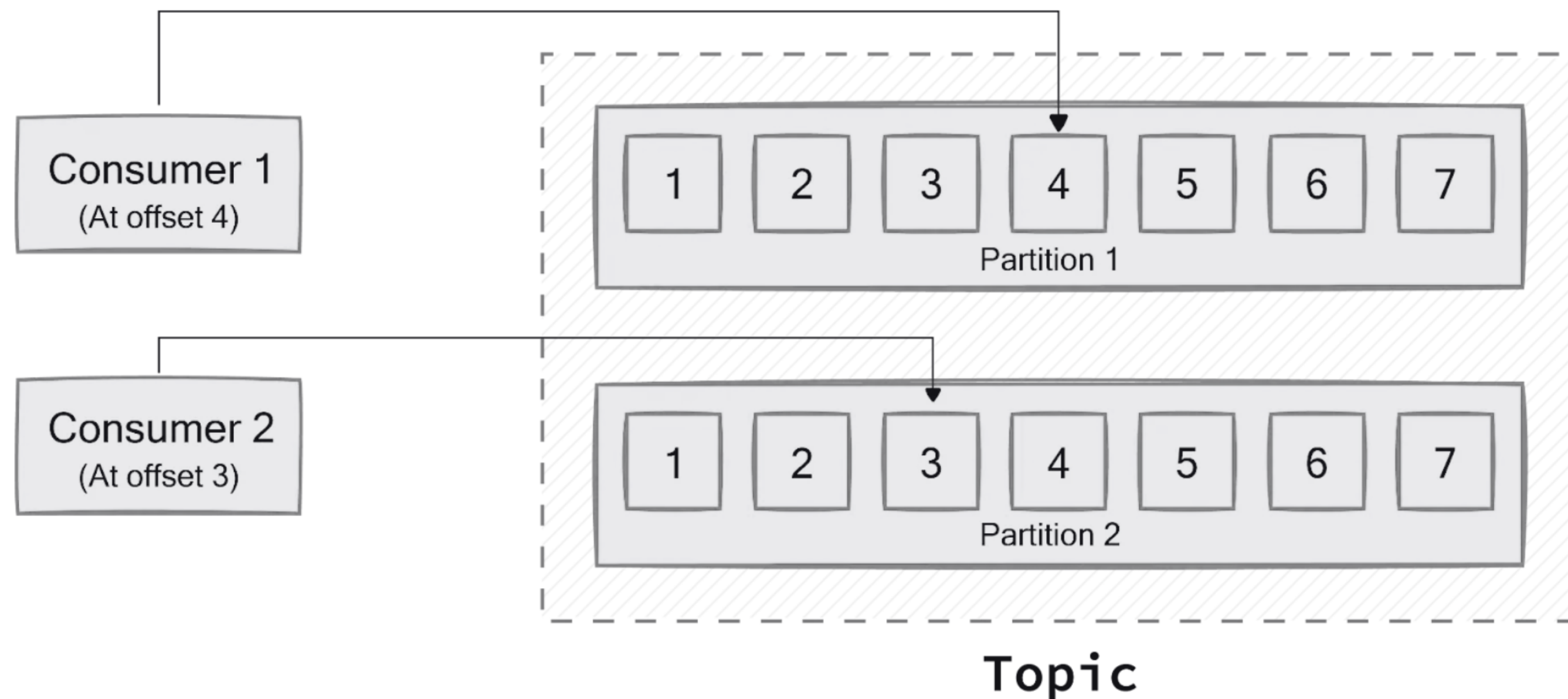
Kafka. Наполнение данными патриции

Kafka позволяет детерминированно назначать сообщения в партии при помощи **ключа партии** (*partition key*). Ключ партии — это часть данных (обычно некоторый атрибут самого сообщения, например, идентификатор), к которой применяется алгоритм для определения партии.

- Назначим ключом партии поле UUID сообщений.
- Продюсер применяет алгоритм (например, как GP модифицирует каждое значение UUID по количеству партий)
- Каждое сообщение помещается в соответствующую партию



Kafka. Смещение



Offset (смещение) — это номер сообщения внутри **partition**.

Kafka не забывает события после чтения. Сообщения не удаляются в течении времени которое указано в настройках кафки. Каждый consumer сам помнит, до какого offset он дочитал.

Проверка подключения и мета информации

Подключение к кластеру с помощью
KafkaCat (kcat)

```
kcat -b  
<внутренний_IP_брокера>:<порт_брокера> \  
-X security.protocol=SASL_PLAINTEXT \  
-X sasl.mechanism=SCRAM-SHA-512 \  
-X sasl.username="<логин_пользователя>" \  
-X sasl.password="<пароль_пользователя>"
```

Проверка метаданных кластера с
помощью KafkaCat (kcat)

```
kcat -b  
<внутренний_IP_брокера>:<порт_брокера> -L \  
-X security.protocol=SASL_PLAINTEXT \  
-X sasl.mechanism=SCRAM-SHA-512 \  
-X sasl.username="<логин_пользователя>" \  
-X sasl.password="<пароль_пользователя>"
```

KafkaCat. Консольная работа с Кафкой

Запись сообщения в Кафку

```
kcat -P \
  -b <внутренний_IP_брокера>:<порт_брокера> \
  -t <имя_топика> \
  -X security.protocol=SASL_PLAINTEXT \
  -X sasl.mechanism=SCRAM-SHA-512 \
  -X sasl.username="<ЛОГИН_ПОЛЬЗОВАТЕЛЯ>" \
  -X sasl.password="<пароль_пользователя>" -K:
```

Далее необходимо внести сообщение в формате **key**:value

1:foo

2:bar

Для завершения ввода необходимо использовать Ctrl + D

Чтение сообщения из Кафки

```
kcat -b <внутренний_IP_брокера>:<порт_брокера> \
  -t <имя_топика> \
  -C -o beginning \
  -X security.protocol=SASL_PLAINTEXT \
  -X sasl.mechanism=SCRAM-SHA-512 \
  -X sasl.username="<ЛОГИН_ПОЛЬЗОВАТЕЛЯ>" \
  -X sasl.password="<пароль_пользователя>"
```

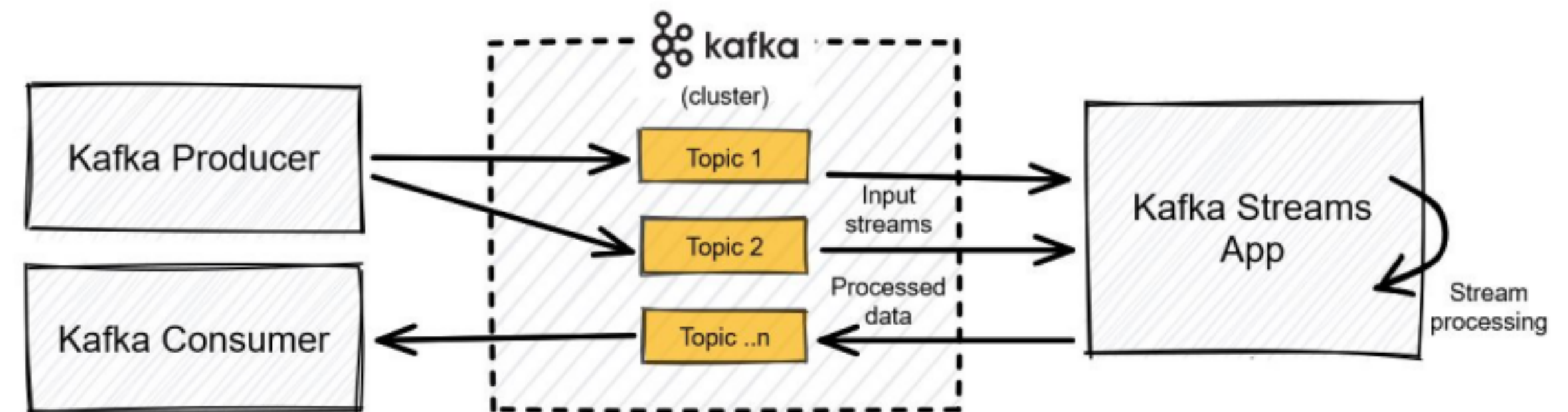

Python. Работа с Кафкой

Запись сообщения в Кафку

```
from kafka import KafkaProducer
producer = KafkaProducer(bootstrap_servers='localhost:1234')
for _ in range(100):
    producer.send('foobar', b'some_message_bytes')
```

Чтение сообщения из Кафки

```
from kafka import KafkaConsumer
consumer = KafkaConsumer('my_favorite_topic')
for msg in consumer:
    print(msg)
```



Для чего применять Kafka Cluster

Для каких задач **можно** применять Кафку:

- Обработка событий или логов
- IoT-приложение для сенсоров
- Доставка событий многим потребителям
- Система уведомлений и событий
- Обработка большого потока данных
- Проектирование event-driven системы

Для каких задач **не стоит** применять Кафку:

- Подмена реляционный или не реляционный СУБД
- Чтение подряд больших сообщений данных
- Нужна гибкая кастомизация каждого сообщения
- Нужны очереди, поддерживающие сложные схемы обработки сообщений